

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №4
по дисциплине «Построение и анализ алгоритмов»
Тема: Поиск подстроки в строке (КМП).

Студент гр. 4345

Наумов М. В.

Преподаватель

Жангиров Т. Р.

Санкт-Петербург

2026

Задание

Реализуйте алгоритм КМП и с его помощью для заданных шаблона P ($|P| \leq 25000$) и текста T ($|T| \leq 5000000$) найдите все вхождения P в T .

Вход:

- Первая строка — P
- Вторая строка — T

Выход:

индексы начал вхождений P в T , разделённые запятой; если P не входит в T , то вывести -1.

Test data			^
# Test	Input data	Output data	
1	ab abab	0, 2	

Заданы две строки A ($|A| \leq 5000000$) и B ($|B| \leq 5000000$).

Определить, является ли A циклическим сдвигом B (это значит, что A и B имеют одинаковую длину и A состоит из суффикса B , склеенного с префиксом B). Например, defabc является циклическим сдвигом abcdef.

Вход:

- Первая строка — A
- Вторая строка — B

Выход:

Если A является циклическим сдвигом B , то индекс начала строки B в A ; иначе вывести -1. Если возможно несколько сдвигов, вывести первый индекс.

Test data			^
# Test	Input data	Output data	
1	defabc abcdef	3	

Описание алгоритма

1. Концептуальные основы алгоритма Кнута-Морриса-Пратта

Фундаментальная задача алгоритма КМП — минимизация избыточных сравнений при поиске подстроки. В отличие от «наивного» алгоритма, который при каждом несовпадении сбрасывает указатель текста назад, КМП реализует принцип однократного прохода. Это достигается за счёт предварительного анализа внутренней структуры самого шаблона (P).

Вместо того чтобы рассматривать текст как случайный набор символов, алгоритм использует уже полученную информацию о совпавших символах, чтобы определить максимально возможный «безопасный» сдвиг шаблона.

2. Префикс-функция: Формирование «карты переходов»

Ключевым инструментом оптимизации является префикс-функция (pref). Для каждой позиции i в шаблоне вычисляется значение $\text{pref}[i]$, которое представляет собой длину самого длинного собственного префикса подстроки $P[0..i]$, являющегося одновременно её суффиксом.

- **Линейность вычислений:** Значение каждой ячейки вычисляется на основе предыдущих состояний с использованием метода двух указателей.
- **Динамика откатов:** Если очередной символ не совпадает, алгоритм не начинает проверку с нуля, а «откатывается» к уже известному меньшему префиксу. Суммарная сложность этого этапа ограничена $O(|P|)$, так как указатель отката в совокупности не может пройти больше шагов, чем было сделано вперед.

3. Механизм поиска и обработка вхождений

В процессе сканирования основного текста (T) указатель движется исключительно слева направо. При обнаружении несоответствия символов алгоритм обращается к массиву `pref`, который служит инструкцией: на какую позицию в шаблоне следует переместить указатель поиска, чтобы продолжить проверку без риска пропустить вхождение.

Для корректного нахождения перекрывающихся вхождений (например, `aba` в строке `ababa`) алгоритм не останавливается после фиксации полного совпадения. Вместо сброса в ноль, состояние обновляется через значение `pref[k-1]`, что позволяет учитывать текущий суффикс как начало возможного нового вхождения.

4. Анализ вычислительной сложности

Временная сложность: $O(|T| + |P|)$

Эффективность алгоритма обусловлена отсутствием вложенных циклов, зависящих друг от друга мультипликативно.

- **Препроцессинг:** Построение вспомогательного вектора за $O(|P|)$.
- **Поиск:** Сканирование текста за $O(|T|)$.

- Константные операции: Сравнение символов и доступ к памяти по индексу выполняются за $O(1)$. Логирование операций выполняется синхронно и не меняет асимптотическую сложность.

Сложность по памяти: $O(|T| + |P|)$

Механизм работы префикс-функции и процесса поиска

Реализованный алгоритм КМП отказывается от ресурсоёмких структур (таких как таблицы переходов конечных автоматов) в пользу итеративного анализа симметрии шаблона. Весь процесс разделен на два этапа, объединенных общей логикой «безопасных откатов».

1. Формирование вектора префикс-функции для строки p

Вычисление значений в массиве pref происходит путём последовательного сканирования искомого образца p . Для этого используются два динамических указателя:

- Указатель i — определяет текущую границу рассматриваемой подстроки (двигается от 1 до $n-1$).
- Указатель j — аккумулирует длину текущего максимального префикса, который одновременно является суффиксом.

Логика переходов:

1. При совпадении символов $p[i]$ и $p[j]$ происходит инкремент j , и результат фиксируется в $\text{pref}[i] = j + 1$. Это означает расширение найденной симметричной области.
2. При обнаружении несовпадения (диссонанса) система инициализирует цикл оптимизированного отката. Указатель j мгновенно перемещается к значению $\text{pref}[j - 1]$. Данный шаг позволяет алгоритму

«вспомнить» более короткие префиксы, уже найденные на предыдущих итерациях, полностью исключая необходимость повторного сравнения ранее пройденных символов.

2. Процедура идентификации вхождений в тексте t

Поиск в исходном тексте t архитектурно повторяет логику построения префикс-функции, что обеспечивает единообразие вычислительной сложности:

1. Линейное сканирование: Указатель i перемещается по тексту t строго транзитивно (только вперед). Это ключевое преимущество КМП: текст считывается один раз, без возвратов назад.

2. Динамическое сопоставление: Указатель j фиксирует прогресс совпадения с шаблоном p . При возникновении расхождения между $t[i]$ и $p[j]$, массив $pref$ предоставляет индекс для совершения «прыжка» назад, минуя заведомо ложные позиции.

3. Фиксация результата: Достижение указателем j величины, равной длине шаблона n , идентифицирует факт полного вхождения. Координата начала вхождения вычисляется как $i - n + 1$ и сохраняется в список `answ`.

3. Обработка перекрывающихся вхождений

Важной особенностью данной реализации является корректная обработка вхождений, накладывающихся друг на друга (например, поиск `aba` в `ababa`).

После успешной идентификации полного совпадения алгоритм не сбрасывает состояние j в ноль. Вместо этого выполняется принудительный откат через $j = pref[-1]$ (или $pref[n-1]$). Это позволяет системе рассматривать суффикс

только что найденного шаблона как префикс возможного следующего вхождения, обеспечивая тем самым максимальную полноту выходных данных за один проход по тексту.

Заметка для отчёта: В контексте задачи на циклический сдвиг, данный механизм поиска применяется к виртуально удвоенному тексту (индексация $t[i \% n]$), что позволяет эффективно находить вхождения шаблона p , «разрезанного» границей строки.

Структуры данных и ключевые переменные:

В реализации алгоритмов в файлах `f5.py` и `KMP.cpp` используются следующие ключевые элементы:

- p и t — строковые переменные, представляющие искомый образец и текст соответственно. В задаче на циклический сдвиг (`KMP.cpp`) t соответствует строке A , а p — строке B . Такое именование позволяет сохранить единообразие логики: p всегда является шаблоном, для которого строится префикс-функция.
- `pref (vector / list)` — основная вычислительная структура данных. Это массив, размер которого равен длине шаблона p . В нём хранятся вычисленные значения префикс-функции, что позволяет алгоритму за константное время $O(1)$ определять длину «отката» при несовпадении символов.
- j (int) — универсальный указатель-счётчик текущей длины совпадения.
 - В обоих файлах на этапе построения `pref` переменная j определяет длину текущего совпадающего префикса.
 - На этапе поиска j указывает на индекс следующего ожидаемого символа в шаблоне p .
- i (int) — индекс основного цикла.
 - В `f5.py` он последовательно проходит по тексту t .

- В КМР.cpp он пробегает диапазон до $2 * \text{len}(t) - 1$, а в сочетании с оператором $\% n$ (остаток от деления) позволяет имитировать поиск в удвоенной строке без выделения дополнительной памяти.
- `answ (list)` — используется только в `f5.py`. Это динамический массив (список), в который записываются индексы всех найденных вхождений. Это необходимо для последующего вывода всех результатов через запятую.
- `log / log2 (ofstream / file object)` — служебные объекты для организации логирования в файлы `log.txt` и `log2.txt`. Они обеспечивают подробную трассировку (запись каждой итерации, условий в циклах `while` и обновлений массива `pref`), позволяя визуализировать работу алгоритма без вывода лишних данных в консоль.

Тестирование

Результаты тестирования представлены в табл. 1 и 2.

Таблица 1 – Результаты тестирования (программа 1, КМП)

№ п/п	Входные данные	Выходные данные	Комментарии
1.	aba abababccabsba	0,2	Простой пример
2.	aaaa aaaaaa	0,1,2	Много вхождений подряд
3.	abc skfofjsodfrkwm	-1	Отсутствие вхождений
4.	abcabc abc	-1	Подстрока длиннее строки
5.	qwerty qwerty	0	Точное совпадение
6.	a asjdcijadoajdaijcosjdcia oia	0,7,10,13,23,26	Поиск одной буквы

Таблица 2 – Результаты тестирования (программа 1, цикл. сдвиг)

№ п/п	Входные данные	Выходные данные	Комментарии
-------	----------------	-----------------	-------------

1.	qwerty yqwert	5	Простой сдвиг
2.	abcdef abcdef	0	Точное совпадение
3.	abc abcd	-1	Разная длина
4.	bacd abcd	-1	Не сдвиг

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Название файла: f5.py

```
log = open("log.txt", "w", encoding="utf-8")
log.write("--- Начало работы алгоритма КМП ---\n")

p = input()
log.write(f"Считан образец (p): '{p}'\n\n")

n = len(p)
pref = n * [0]

log.write("--- Построение префикс-функции ---\n")
log.write(f"Инициализация массива pref нулями. Длина массива: {n}\n")

for i in range(1, n):
    log.write(f"\nИтерация i={i}, текущий символ p[{i}] = '{p[i]}'\n")
    j = pref[i - 1]
    log.write(f"    Начальное значение j (из pref[{i - 1}]): {j}\n")

    while p[i] != p[j] and j > 0:
        log.write(f"        Несовпадение: p[{i}] ('{p[i]}') != p[{j}] ('{p[j]}')\n")
        log.write(f"        Откат j к pref[{j - 1}]\n")
        j = pref[j - 1]
        log.write(f"        Новое значение j = {j}\n")

    if p[i] != p[j]:
        log.write(f"        Совпадений нет, j остается равным {j}\n")
        pref[i] = 0
        log.write(f"        -> Записываем pref[{i}] = 0\n")
    else:
        log.write(f"        Совпадение: p[{i}] == p[{j}] ('{p[i]}'). Увеличиваем j.\n")
        pref[i] = j + 1
        log.write(f"        -> Записываем pref[{i}] = {j + 1}\n")

    log.write(f"    Текущий массив pref: [{', '.join(map(str, pref))}]\n")

t = input()
log.write(f"\nСчитан текст (t): '{t}'\n")

log.write("\n--- Поиск вхождений образца в тексте ---\n")
j = 0
log.write(f"Начальная длина совпадения j = 0\n")
answ = []

for i in range(len(t)):
    log.write(f"\nИтерация по тексту i={i}, текущий символ t[{i}] = '{t[i]}'\n")
    log.write(f"    Текущая длина совпадения j = {j}\n")

    while t[i] != p[j] and j > 0:
        log.write(f"        Несовпадение: t[{i}] ('{t[i]}') != p[{j}] ('{p[j]}')\n")
        log.write(f"        Откат j к pref[{j - 1}]\n")
        j = pref[j - 1]
        log.write(f"        Новое значение j = {j}\n")

    if t[i] == p[j]:
        log.write(f"        Совпадение: t[{i}] == p[{j}] ('{t[i]}'). Увеличиваем
```

```

j.\n")
    j += 1
    if j == n:
        log.write(f"    !!! Найдено полное вхождение образца (j == {n}).
Индекс начала: {i - n + 1}\n")
        answ.append(str(i - n + 1))
        log.write("    Откат j для поиска следующих вхождений к pref[-
1]\n")
        j = pref[-1]
        log.write(f"    Новое значение j = {j}\n")
    else:
        log.write(f"    Совпадений нет, j остается равным {j}\n")

log.write("\n--- Завершение алгоритма ---\n")
log.write(f"Всего найдено вхождений: {len(answ)}\n")
log.close()

if answ:
    print(', '.join(answ))
else:
    print(-1)

```

Название файла: КМР.cpp

```

#include <iostream>
#include <string>
#include <vector>
#include <fstream>
int main() {
    std::ofstream log("log2.txt");
    log << "--- Проверка на циклический сдвиг (алгоритм КМП) ---\n";

    std::ios_base::sync_with_stdio(false);
    std::cin.tie(NULL);

    std::string t, p;
    if (!(std::cin >> t >> p)) {
        return 0;
    }

    log << "Считана строка A (t): " << t << "\n";
    log << "Считана строка B (p): " << p << "\n\n";

    int n = p.length();
    int m = t.length();

    log << "Длина строки A (m) = " << m << "\n";
    log << "Длина строки B (n) = " << n << "\n\n";

    log << "--- Построение префикс-функции для строки p ---\n";
    std::vector<int> pref(n, 0);
    log << "Инициализация массива pref нулями. Длина массива: " << n << "\n";

    for (int i = 1; i < n; ++i) {
        log << "\nИтерация i=" << i << ", текущий символ p[" << i << "] = " << p[i] << "\n";
        int j = pref[i - 1];
    }
}

```

```

log << " Начальное значение j (из pref[" << i - 1 << "]): " << j << "\n";

while (p[i] != p[j] && j > 0) {
    log << " Несовпадение: p[" << i << "] (" << p[i] << ") != p[" << j << "] (" << p[j] << ") \n";
    log << " Откат j к pref[" << j - 1 << "] \n";
    j = pref[j - 1];
    log << " Новое значение j = " << j << "\n";
}
if (p[i] != p[j]) {
    log << " Совпадений нет, j остается равным " << j << "\n";
    pref[i] = 0;
} else {
    log << " Совпадение: p[" << i << "] == p[" << j << "] (" << p[i] << "). Увеличиваем j. \n";
    pref[i] = j + 1;
}
log << " -> Записываем pref[" << i << "] = " << pref[i] << "\n";

log << " Текущий массив pref: [";
for (int k = 0; k < n; ++k) {
    log << pref[k] << (k == n - 1 ? "" : ", ");
}
log << "] \n";
}

if (n != t.length()) {
    log << "Длины строк не совпадают (n != m). Циклический сдвиг невозможен. Возврат -1. \n";
    log.close();
    std::cout << -1 << "\n";
    return 0;
}

log << "\n--- Поиск первого вхождения p в виртуальный текст t+t (КМП) --- \n";
int j = 0;
log << "Начальная длина совпадения j = 0 \n";

for (int i = 0; i < (2 * t.length() - 1); ++i) {
    log << "\nИтерация по тексту i=" << i << ", текущий символ t[" << i << " % " << n << "] = " << t[i % n] << " \n";
    log << " Текущая длина совпадения j = " << j << "\n";

    while (t[i % n] != p[j] && j > 0) {
        log << " Несовпадение: t[" << i << " % " << n << "] (" << t[i % n] << ") != p[" << j << "] (" << p[j] << ") \n";
        log << " Откат j к pref[" << j - 1 << "] \n";
        j = pref[j - 1];
        log << " Новое значение j = " << j << "\n";
    }
    if (t[i % n] == p[j]) {
        log << " Совпадение: t[" << i << " % " << n << "] == p[" << j << "] (" << t[i % n] << "). Увеличиваем j. \n";
        j += 1;
    }
}

```

```

        if (j == n) {
            log << " !!! Найдено полное вхождение строки p (j == " << n << "). Индекс начала (сдвиг): "
<< (i - n + 1) << "\n";
            log << "--- Завершение алгоритма (успех) ---\n";
            log.close();
            std::cout << i - n + 1 << "\n";
            return 0;
        }
    } else {
        log << " Совпадений нет, j остается равным " << j << "\n";
    }
}

log << "\nВхождение строки p в циклическую строку t не найдено.\n";
log << "--- Завершение алгоритма (неудача) ---\n";
log.close();
std::cout << -1 << "\n";
return 0;
}

```